

CodeHogwarts — гайд по проекту

Короткая карта: что это, на чём написано, где что лежит и как оно работает. Читается за десять минут, дальше можно возвращаться к нужному разделу.

Живой стенд: <https://maks.my/codehog/>

1. Что это

Тренажёр Python в виде игры. Человек заходит без регистрации, отвечает, программировал ли раньше, проходит тест на шесть вопросов, получает курс под свой уровень и решает задания прямо в браузере. Код не проверяется по образцу, а **реально исполняется на сервере**. За пройденные уроки начисляются монеты, каждые пять уроков даётся вращение колеса, на монеты покупаются скины ежа.

Скинов 54: шесть уровней по девять. Первый уровень открыт сразу, из него при регистрации бесплатно выбирается стартовый ёж. Каждый пройденный модуль курса открывает следующий уровень.

Смысл в одной фразе: **выполненный код превращается в коллекцию ежей**.

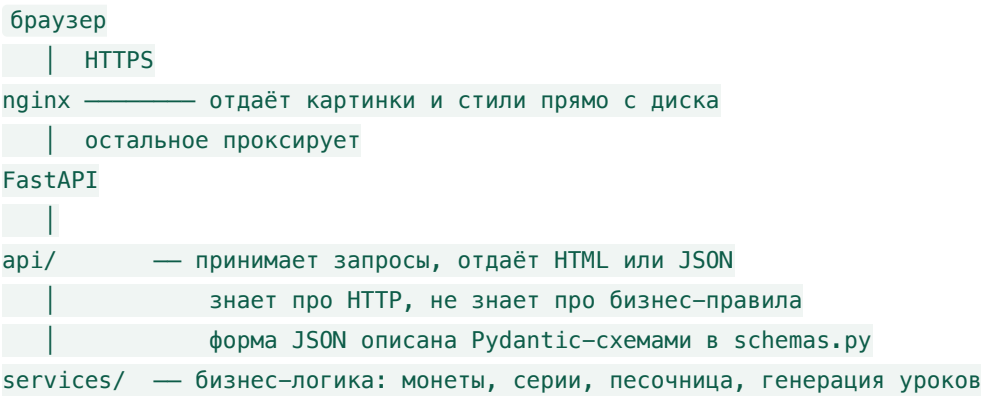
2. На чём написано

Слой	Технология	Зачем именно она
Сервер	Python 3.14 + FastAPI	Приложение учит Python и запускает код учеников. На той же платформе это делается штатными средствами
Схемы API	Pydantic	Описывают JSON запросов и ответов. FastAPI по ним проверяет входные данные и строит документацию
Работа с базой	SQLAlchemy 2.0	ORM: таблицы описаны классами, SQL руками не пишется
База	SQLite	Один файл. Доступ только через ORM, переезд на PostgreSQL меняет строку подключения
Страницы	Jinja2	HTML собирается на сервере и приходит готовым
Клиент	Ванильный JavaScript	Семь файлов, около 500 строк. Ни React, ни npm, ни сборки
Пароли	bcrypt	Хранятся только хэши
Сессии	itsdangerous	Подписанная cookie, подделать нельзя
Нейросеть	OpenAI-совместимый шлюз	Генерация уроков и подсказок. Модель <code>deepseek-v4-flash</code>
Журнал	logging + journald	Время, уровень, адрес запроса, пользователь и стек каждого сбоя
Развёртывание	systemd + nginx	Свой сервер, HTTPS

Ванильный JavaScript значит «чистый, без фреймворка». Браузер получает файл и сразу исполняет. Ничего не нужно собирать перед выкладкой.

3. Как устроено

Запрос идёт сверху вниз, каждый слой знает только соседний.



```
| не знает, что существует HTTP
models/ — описание таблиц
|
SQLite
```

Правило простое: `services` никогда не импортирует `FastAPI`, а `models` никогда не импортирует `services`. Поэтому логику можно проверять без поднятия сервера.

Отдельно в стороне стоят две вещи:

- **песочница** — отдельный процесс, в котором выполняется код ученика;
- **шлюз нейросети** — внешний сервис, к которому ходят за уроками и подсказками.

4. Где что лежит

```
app/
  main.py      точка входа: собирает приложение, поднимает базу
  config.py    все настройки из переменных окружения
  database.py  подключение и сессии

models/        таблицы, 16 штук
  user.py      пользователь, дни занятий, движение монет
  learning.py  курс, модуль, урок, задание, решение, попытка теста
  game.py      предметы, инвентарь, вращения колеса
  reward.py    медали и мемы
  admin.py     массовые начисления

api/           слой HTTP
  pages.py     страницы: лендинг, тест, урок, магазин, профиль, правила
  actions.py   JSON API под префиксом /hog
  schemas.py   Pydantic-схемы запросов и ответов этого API
  admin.py     панель администратора
  deps.py      зависимости (кто вошёл), шаблонизатор, контекст

services/      бизнес-логика
  economy.py   ЕДИНСТВЕННАЯ точка движения монет, открытие уровней скинов
  sandbox.py   исполнение кода ученика
  course.py    генерация уроков и их проверка
  awards.py    выдача медалей, подбор мемов
  auth.py      регистрация, вход, сессии
  curriculum.py каркас программы, вводные модули для новичков
  ready_lessons.py готовые уроки: вводные модули и первый урок уровня
  seed.py      каталог скинов: 6 уровней × 9, цены
  migrate.py   добавляет новые колонки в существующую базу
  testbank.py  вопросы вводного теста
  ai.py        клиент нейросети
  i18n.py      переводы интерфейса

templates/     HTML-шаблоны
static/
```

css/	один файл стилей
js/	скрипты страниц, errors.js – окно ошибки
img/logo/	логотип и значок вкладки
img/skins/	54 скина ежа: <уровень>--<номер>.webp
img/memes/	мемы, которые выпадают за урок

docs/	документация и сборщики Word и схемы
диаграммы/	картинки для документа
задание/	методичка кафедры и юз-кейсы
deploy/	systemd и nginx
data/	файл базы (в репозиторий не попадает)

5. Механизмы, которые стоит понимать

Исполнение кода ученика

Код не разбирается и не сравнивается с образцом. Он **запускается** в отдельном процессе с тремя ограничениями: не дольше пяти секунд, не больше 128 мегабайт памяти, не больше десяти тысяч символов вывода. Процесс стартует без доступа к переменным окружения и в пустом временном каталоге. Результат возвращается через служебную метку в выводе.

Перед запуском код проходит грубую проверку на запрещённые модули и вызовы. Это не полноценная изоляция, и в документации это признано прямо: для публичного сервиса нужен контейнер без сети.

Проверка сгенерированных уроков

Самая ценная идея в проекте. Нейросеть выдаёт урок вместе с эталонным решением и тестами. Прежде чем урок попадёт к ученику, **эталонное решение запускается в песочнице против этих же тестов**. Не сошлось — урок отбрасывается целиком и берётся шаблонный.

То есть враньё модели физически не доезжает до пользователя.

Первый урок курса и оба вводных модуля написаны вручную (`services/ready_lessons.py`) и модели не отдаются: первый урок должен открываться мгновенно, а новичкам нужны выверенные задания.

Экономика через одну точку

Любое изменение баланса проходит через один метод. Он записывает строку в журнал операций и запрещает уйти в минус. Поле с балансом существует только ради скорости чтения: настоящий баланс всегда можно пересчитать суммой по журналу и сверить.

Защита от двойных начислений

Двойной клик по колесу или две открытые вкладки не дают приз дважды. Работает не проверка в коде, а **уникальное ограничение в базе** на пару «пользователь и веха». Второй запрос упирается в него и отваливается. Проверку в коде можно обойти параллельным запросом, ограничение базы нельзя.

Так же защищены медали и задания внутри урока.

Путь новичка

Перед тестом один вопрос: программировал ли раньше. Три ответа — три пути:

Ответ	Что происходит
Никогда	Тест пропускается, уровень «новичок», курс начинается с модуля «Что такое программа»
На другом языке	Тест проходит, а в курс первым встаёт модуль «Python после другого языка»
На Python	Обычный тест и курс по его итогам

Ответ хранится в `test_attempts.experience`, вводные модули описаны в `services/curriculum.py`.

Скины по уровням

У каждого скина в таблице `items` есть поле `tier` — номер уровня. Открытый уровень не хранится, а считается: один плюс число полностью пройденных модулей. Поэтому его нельзя «сломать» рассинхронизацией. Купить скин закрытого уровня нельзя: отказ идёт из `economy.py`, а не только из интерфейса. Старая одежда не удалена из базы, а выключена флагом `is_active`, чтобы не потерять историю покупок.

Журнал ошибок

Любое необработанное падение перехватывает middleware в `main.py`. В журнал пишется время, адрес запроса, ID пользователя и полный стек, а пользователь видит короткое сообщение и кнопку «Повторить» (`static/js/errors.js`). Стек наружу не уходит. Смотреть журнал на сервере: `journalctl -u codehog`.

Работа без нейросети

Убери ключ из настроек, и приложение продолжит работать: уроки станут шаблонными, подсказки статичными, всё остальное не изменится. Внешний сервис не является точкой отказа.

6. Как здесь используется FastAPI

Сайт отдаёт два вида ответов, и для каждого FastAPI применён по назначению.

Страницы (`api/pages.py`) — HTML из шаблонов Jinja2. Формы регистрации и входа принимаются через `Form(...)`, после отправки идёт редирект. Это классический серверный рендеринг, FastAPI его поддерживает штатно.

JSON API (`api/actions.py`, префикс `/hog`) — то, к чему ходит JavaScript: тест, проверка заданий, подсказки, колесо, магазин. Здесь всё, ради чего FastAPI задуман:

- **Pydantic-схемы** (`api/schemas.py`). Тело запроса объявлено классом, например `TaskAnswer`. Неверный JSON отклоняется с кодом 422 ещё до нашего кода, а ответ описан через `response_model` — наружу уходят только объявленные поля.
- **Зависимости** (`Depends`). Сессия базы, текущий пользователь и проверка входа (`нужен_пользователь` — без входа 401) подключаются параметром, а не копируются в каждую функцию.
- **Автодокументация**. Swagger со всеми схемами открывается по адресу `https://maks.my/codehog/hog/docs`. Ручки разбиты по тегам: тест, уроки, колесо, магазин.
- **Lifespan**. Подготовка базы при старте сделана через `lifespan`, а не через устаревший `on_event`.
- **Обработчики исключений**. Ошибка проверки данных, отказ «не хватает монет» и неожиданный сбой приходят в одном формате `{"detail": "..."}.`

Классы и поля схем названы по-английски, хотя остальной код кириллический. Это внешний контракт: поля — ключи JSON для фронтенда, а генератор OpenAPI пропускает в имена схем только латиницу.

7. Как запустить и выкатить

Локально:

```
./run.sh # поднимет окружение и запустит
```

Откроется на `http://127.0.0.1:8000/codehog/`. Подкаталог задаётся переменной `BASE_PATH` в файле `.env`.

На сервер:

```
rsync -az --delete --exclude '.venv' --exclude 'data' --exclude '.env*' \
--exclude '.git' --exclude '__pycache__' ./ root@maks.my:/opt/codehog/
ssh root@maks.my 'chown -R www-data:www-data /opt/codehog && systemctl restart codehog'
```

Схема базы обновляется сама при старте (`services/migrate.py`), отдельных миграций запускать не нужно. Перед выкладкой стоит скопировать файл базы: `cp data/codehog.db data/codehog-$(date +%F).db`.

8. Куда смотреть при типовых задачах

Надо сделать	Файл
Изменить награды за урок или цены	<code>services/economy.py</code> , <code>services/seed.py</code>
Добавить скин или уровень	картинка в <code>static/img/skins/</code> , строка в <code>services/seed.py</code>
Поменять правило открытия уровней	<code>services/economy.py</code> — <code>открытый_уровень</code>
Добавить поле в JSON-ответ	<code>api/schemas.py</code> , затем <code>api/actions.py</code>
Заменить логотип	<code>static/img/logo/</code>
Добавить тему в курс	<code>services/curriculum.py</code>
Поправить готовый урок	<code>services/ready_lessons.py</code>
Изменить, что генерирует нейросеть	<code>services/course.py</code> — промпт и проверка
Добавить тип задания	<code>models/learning.py</code> , <code>services/course.py</code> , <code>api/actions.py</code> , <code>templates/lesson.html</code>
Добавить медаль	<code>services/awards.py</code> — каталог наверху файла
Заменить подборку мемов	картинки в <code>static/img/memes/</code> , список в <code>services/awards.py</code>
Добавить язык	<code>services/i18n.py</code>
Поменять внешний вид	<code>static/css/style.css</code> — токены в начале файла
Пересобрать схему базы для документации	<code>docs/собрать-схему.py</code>
Пересобрать документ в Word	<code>docs/собрать-документ.py</code>

9. Что честно не сделано

1. **Автотестов нет.** Проверки ручные и скриптами, воспроизводимые, но `pytest` это не заменяет.
2. **Изоляция кода процессная, не контейнерная.** Для учебного стенда достаточно, для публичного сервиса мало.
3. **Раздела CMS нет.** Уроки и скины задаются в коде, хотя медали и мемы уже вынесены в таблицы.